

2026년도 클라우드컴퓨팅 직종 연습 과제

직 종 명	클라우드 컴퓨팅	과 제 명	AWS Deep-Dive Challenge	과 제 번 호	제 2과 제
경기시간	2시간	비 번 호		심 사 위 원 확 인	(인)

1. 요구사항

-
1. 이벤트 드리븐 서버리스 파이프라인

2. 유의사항

-
1. 문제에 제시된 괄호 `<>` 는 변수를 뜻하므로 선수가 적절히 변경하여 사용해야 합니다.
2. Security Group의 80/443 outbound는 any-open으로 사용할 수 있도록 합니다.
3. 과제 종료 전 실행 중인 테스트 및 부하를 중지하여 서버에 문제가 없도록 해야 합니다.
4. 모든 리소스의 이름, 태그, 변수는 **대소문자를 구분**합니다.
5. 태그 정보가 맞지 않거나 권한 문제가 생겨 채점이 불가하지 않도록 주의합니다.
6. 문제지에서 제공되지 않는 값은 AWS Well-Architected Framework 6 Pillars를 참고하여 적절한 값으로 구성해야 합니다.
7. CloudShell에서 채점 스크립트를 실행해 채점을 진행합니다.
8. 경기 시작 전 기본 서비스들은 모두 삭제합니다.
9. 과제 비용이 \$50를 초과하지 않도록 리소스를 최소화하여 구성합니다.
10. Amazon Q CLI를 최소 1회 사용하여 과제를 수행합니다.

No 1. 이벤트 드리븐 서버리스 파이프라인

해당 과제 풀이는 **서울(ap-northeast-2)** 리전에서 진행합니다.

전자상거래 플랫폼의 주문 처리 백엔드를 Lambda 없이 서버리스로 구축합니다. 클라이언트가 API Gateway로 주문을 제출하면 SQS에 적재되고, EventBridge Pipes가 SQS 메시지를 Step Functions Express 워크플로우로 직접 전달합니다. Step Functions은 AWS SDK 직접 통합(optimistic integration)으로 DynamoDB, S3, EventBridge를 호출하여 Lambda 없이 전체 파이프라인을 처리합니다. 고액 주문(price ≥ 300,000)은 S3에 아카이브하고, 그 외는 DynamoDB에 저장합니다. 처리 결과는 EventBridge 커스텀 이벤트 버스로 발행하며, CloudWatch 알람과 대시보드로 운영 가시성을 확보합니다.

1. DynamoDB

주문 데이터를 저장할 테이블과 처리 상태를 관리할 상태 테이블을 각각 생성합니다. 두 테이블 모두 On-Demand 과금 방식을 사용하고, 삭제 방지(Deletion Protection)를 활성화합니다. orders 테이블에는 `orderId` (String) → `status` (String) 방향의 GSI를 추가하여 상태별 조회가 가능하도록 합니다. Point-in-Time Recovery를 활성화합니다.

- Orders Table Name : `wsc2026-orders`
- Orders Partition Key : `orderId` (String)
- Orders Sort Key : `createdAt` (String)
- GSI Name : `status-index` (PK: `status`, SK: `createdAt`)
- State Table Name : `wsc2026-order-state`

- State Partition Key : `executionId` (String)

2. S3

고객 주문 아카이브용 버킷을 생성합니다. 서버 사이드 암호화(SSE-KMS, AWS 관리형 키)를 적용하고, 퍼블릭 액세스를 완전히 차단합니다. 버전 관리를 활성화하며, 30일 후 객체를 S3 Intelligent-Tiering으로 전환하는 수명 주기 규칙을 구성합니다.

- Archive Bucket Name : `wsc2026-orders-archive-<비번호>`
- Archive Object Key 형식 : `orders/YYYY/MM/DD/<orderId>.json`

3. SQS

주문 수신 큐(표준)와 데드 레터 큐를 생성합니다. 메인 큐의 최대 수신 횟수는 3으로 설정하고, 3회 초과 시 DLQ로 메시지가 이동되도록 구성합니다. 메시지 보관 기간은 메인 큐 4일, DLQ 14일로 설정합니다. 큐에 저장되는 메시지를 SSE-SQS로 암호화합니다.

- Main Queue Name : `wsc2026-order-queue`
- DLQ Name : `wsc2026-order-dlq`
- Visibility Timeout : 300초

4. EventBridge

주문 처리 결과 이벤트를 라우팅할 커스텀 이벤트 버스와, SQS를 Step Functions에 연결할 Pipe를 생성합니다.

커스텀 이벤트 버스: Step Functions이 이벤트 버스에 `source: wsc2026.orders`, `detail-type: OrderProcessed` 또는 `OrderFailed` 형태로 이벤트를 발행합니다. 이벤트 패턴 기반 Rule을 두 개 생성합니다. 성공 Rule은 `detail-type: OrderProcessed`를 필터링하여 DynamoDB `wsc2026-order-state` 테이블을 타겟으로 지정합니다 (SDK 직접 통합). 실패 Rule은 `detail-type: OrderFailed`를 필터링하여 SQS DLQ를 타겟으로 지정하고, Input Transformer로 `{"source": "eventbridge", "orderId": "<$.detail.orderId>", "reason": "<$.detail.reason>"}` 형식으로 변환하여 전송합니다. 이벤트 아카이빙을 활성화하여 최근 7일간의 모든 이벤트를 보존합니다.

EventBridge Pipes: SQS 메인 큐를 소스로, Step Functions Express 상태 머신을 타겟으로 하는 Pipe를 생성합니다. Pipe의 배치 크기는 1로 설정하고, SQS 메시지 본문을 상태 머신 입력으로 그대로 전달합니다.

- Event Bus Name : `wsc2026-order-bus`
- Success Rule Name : `wsc2026-rule-success`
- Failure Rule Name : `wsc2026-rule-failure`
- Archive Name : `wsc2026-order-archive`
- Archive Retention : 7일
- Pipe Name : `wsc2026-order-pipe`
- Pipe IAM Role : `wsc2026-pipe-role`

5. Step Functions

주문 처리 흐름을 관리하는 Express 타입 상태 머신을 생성합니다. Lambda를 사용하지 않고 AWS SDK 직접 통합(optimistic integration)으로 각 AWS 서비스를 호출합니다. 상태 머신은 다음 상태로 구성합니다.

ValidateOrder (Choice) — 입력 데이터에 `orderId`, `product`, `quantity`, `price`, `timestamp` 필드가 모두 존재하는지 조건을 검사합니다. 조건 불충족 시 **HandleError**로 분기합니다.

CheckPrice (Choice) — `$.price`가 300,000 이상이면 **ArchiveToS3**로, 미만이면 **SaveToDynamoDB**로 분기합니다.

ArchiveToS3 (Task) — `arn:aws:states:::aws-sdk:s3:putObject`를 사용하여 아카이브 버킷의 `orders/YYYY/MM/DD/<orderId>.json` 경로에 저장합니다. 완료 후 **NotifySuccess**로 이동합니다.

SaveToDynamoDB (Task) — `arn:aws:states:::dynamodb:putItem`을 사용하여 `wsc2026-orders` 테이블에 저장합니다. 완료 후 **NotifySuccess**로 이동합니다.

NotifySuccess (Task) — `arn:aws:states:::events:putEvents`를 사용하여 `wsc2026-order-bus`에 `OrderProcessed` 이벤트를 발행합니다.

HandleError (Task) — `arn:aws:states:::events:putEvents` 를 사용하여 `wsc2026-order-bus` 에 `OrderFailed` 이벤트를 발행합니다. 각 Task 상태에 `Catch` 블록을 추가하여 오류 발생 시 이 상태로 분기합니다.

실행 로그를 CloudWatch Logs에 ALL 레벨로 저장합니다. Step Functions IAM Role에는 DynamoDB PutItem, S3 PutObject, EventBridge PutEvents 권한을 최소 범위로 부여합니다.

- State Machine Name : `wsc2026-order-workflow`
- Type : EXPRESS
- IAM Role Name : `wsc2026-sfn-role`
- Log Group : `/aws/states/wsc2026-order-workflow`

6. API Gateway

REST API를 생성하고 주문 제출 엔드포인트를 구성합니다. `POST /orders` 메서드를 생성하여 SQS SendMessage 액션과 AWS 서비스 통합(non-proxy)으로 연결합니다. API Key 인증을 적용하고 사용량 계획(Usage Plan)으로 초당 10 요청, 일일 1,000 요청으로 제한합니다. 요청 본문 검증(Request Validator)을 활성화하여 `orderId`, `product`, `quantity`, `price` 필드가 포함된 모델을 정의합니다. 스테이지 이름은 `prod` 로 배포합니다.

- API Name : `wsc2026-order-api`
- Stage : `prod`
- Usage Plan Name : `wsc2026-order-plan`
- API Key Name : `wsc2026-order-key`
- Model Name : `OrderRequest`

7. CloudWatch

DLQ 메시지 깊이(ApproximateNumberOfMessagesVisible)가 1 이상이 되면 CloudWatch 알람이 트리거되어 EventBridge 커스텀 이벤트 버스에 경보 이벤트를 전송하도록 구성합니다. CloudWatch 대시보드를 생성하여 다음 지표를 시각화합니다: API Gateway 5XX 에러율, SQS DLQ 메시지 수, Step Functions 실행 성공/실패 수, EventBridge Rule 매칭 횟수(성공/실패 Rule 각각).

- Alarm Name : `wsc2026-dlq-depth-alarm`
- Dashboard Name : `wsc2026-order-dashboard`